# University of Twente

Nth time’s the charm

**Index** Page numbers, then formula codes.


| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Arithmetic, Python | **21** | Caching | **23** | Dictionaries / sets | **22** | Gcd pair sums | **24 D2** | Möbius / inversion | **23 D1** | Sorting / binary search | **22** |
| Arrays / lists | **21** | Calendar / dates | **23** | Diophantine equations | **23 N1** | Generating functions | **25 S4** | Necklaces | **24 C4** | Spanning trees | **24 G6** |
| Bell numbers | **24 C1** | Combinatorial iterators | **22** | Divisor sums | **23–24 D1–3** | Geometric distribution | **25 P4** | Pair distances / sums | **25 S2** | Stirling numbers | **24 C1** |
| Big integers | **21** | Complexities, Python | **23** | Exact modular division | **25 B5** | Heaps / merging | **23** | Permutation cycles | **24 C2** | Strings / regex | **22** |
| Binomial identities | **25 B1** | Congruences / CRT | **23 N4** | Expectation | **25 P1** | Hypergeometric | **25 P5** | Planar graphs | **24 G7** | Topological order | **23** |
| Binomial probability | **25 P3** | Coprime pairs | **24 D2** | Exponent reduction | **23 N3,5** | Input / output | **21** | Probability tails | **25 P2** | Totient, Euler $\varphi$ | **23 N2** |
| Binomial, prime powers | **25 B4** | Covariance | **25 P1** | Factorials / zeros | **25 B2** | Inverses, modular | **23 N3** | Prüfer / labeled trees | **24 G5** | Triangles | **24 G2** |
| Bitmasks | **22** | Cycles in graphs | **24 G1–4** | Fibonacci | **25 S3** | JSON | **23** | Python setup | **21 U1–2** | Variance | **25 P1** |
| Bracelets | **24 C4** | Debug / run helpers | **21 U1** | Floor quotients | **24 D3** | Lucas theorem | **25 B3** | Queues / counters | **22** | Walks, adjacency | **24 G3** |
| Burnside / orbits | **24 C3** | Decimals / fractions | **22** | Four-cycles | **24 G3** | Markov / Chebyshev | **25 P6** | Recurrences | **25 S3–4** | Weighted sums | **25 S1** |


**Notation:** $\sum$ sum; $\prod$ product; $a \mid b$: $a$ divides $b$; $\gcd$: greatest common divisor; $\operatorname{lcm}$: least common multiple; $\lfloor x \rfloor$: round down; $\lceil x \rceil$: round up; $\binom{n}{k}$: choose $k$ of $n$; $[P]$: 1 if statement $P$ holds, else 0.

## Python setup

### U1 Create, check and run a solution

Practice: `bash setup.sh`, then a new Bash/zsh terminal. Contest: save U2 as `~/template.py`; paste:

```
mkp() { mkdir -p "$1" && cp -n ~/template.py "$1/$1.py"; }
cpy() {
    python3 -c 'import sys
p = sys.argv[1]
compile(open(p, "rb").read(), p, "exec")' "$1/$1.py"
}
rp() {
    cpy "$1" || return
    if [ -n "${2:-}" ]; then
        LOCAL=1 python3 "$1/$1.py" < "$1/$2.in"
    else LOCAL=1 python3 "$1/$1.py"
    fi
}
```

`mkp a`: create `a/a.py`; `cpy a`: syntax check; `rp a`: run; `rp a 1`: read `a/1.in`. C++: `mk a, c a, r a [1]` (original setup).

### U2 Python template / terminal debugging

```
import os
import sys
input = sys.stdin.buffer.readline
LOCAL = os.environ.get("LOCAL") == "1"

def dbg(*args):
    if LOCAL:
        print(*args, file=sys.stderr)

def solve():
    n = int(input())
    print(n)

if __name__ == "__main__":
    solve()
```

`rp` enables `dbg` on stderr; `rp a 1 > a/1.out` saves output, keeps debug in terminal. Submit `a/a.py`; ordinary `python3` leaves debug off. Buffered input returns bytes.

# 1 Essentials

Standard library only. Run: `python3 sol.py < 1.in`.

## 1.1 Input and output

```
import sys
input = sys.stdin.readline
n = int(input())
a = list(map(int, input().split()))
print(sum(a))
print(*a)                          # space-separated
```

Whole-file integer input (not for interactive problems):

```
import sys
it = iter(map(int, sys.stdin.buffer.read().split()))
n = next(it)
a = [next(it) for _ in range(n)]
```

`buffer` gives bytes; use `token.decode()` for text. Interactive: `print("?", x, flush=True)`.

## 1.2 Arithmetic and comparisons

```
7 / 3                 # float: 2.333...
-7 // 3, -7 % 3       # (-3, 2): division floors
a, b = -7, 3          # b > 0
ceil_div = -(-a // b) # -2, works for either sign of a
2 ** 10               # 1024; ^ is XOR, not power
10 ** 18              # exact int; 1e18 is a float
x, y = 3, 8; x, y = y, x  # swap; unpack a tuple
0 <= x < 10           # chained comparison
x == y                # value equality; "is": identity
```

## 1.3 Lists, slices and grids

```
a = [10, 20, 30, 40]
a.append(50); a.extend([60, 70])
last = a.pop()                  # removes last
a[-1], a[1:3], a[::-1]          # 60, [20,30], reversed
b = a.copy()                    # shallow copy
a[1:3] = [9]                    # replace a slice
n, m = 3, 4
grid = [[0] * m for _ in range(n)]
squares = [x*x for x in range(6) if x % 2 == 0]
```

`a=b` aliases; `[[0]*m]*n` repeats the same row. Slices copy and exclude the stop index.

## 1.4 Loops and built-ins

```
a = [3, 1, 4]
list(range(2, 8, 2))            # [2, 4, 6]
list(enumerate(a))              # [(0,3),(1,1),(2,4)]
list(zip(a, [10, 20]))          # stops at shorter input
len(a), sum(a), min(a), max(a)  # 3, 8, 1, 4
any(x < 0 for x in a)           # False
all(x > 0 for x in a)           # True
list(reversed(a))               # [4, 1, 3]
```

`map`/`zip`/`reversed` are one-pass iterators; `list(...)` stores their results.

# 2 Exact numbers

## 2.1 Huge integers and modular arithmetic

```
a = 10 ** 100                   # arbitrary-size int
q, r = divmod(a, 97)            # a == q*97 + r
int("ff", 16), int("101", 2)    # 255, 5
bin(13), hex(255)               # '0b1101', '0xff'
pow(2, 1000, 97)                # 2**1000 mod 97
pow(3, -1, 10)                  # 7: inverse, gcd=1
abs(-123), len(str(a))          # 123, 101 digits
```

`pow(a,e,m)` avoids building `a**e`. Inverses need coprime `a,m`; `m` need not be prime.

## 2.2 Integer math and counting

```
from math import gcd, lcm, isqrt, comb, perm, factorial
gcd(12, 18, 30), lcm(4, 6, 10)    # 6, 60
isqrt(10 ** 100 + 1)              # 10**50, exact floor
comb(100, 50)                     # exact n choose k
perm(5, 2), factorial(5)          # 20, 120
from math import inf, pi, hypot, atan2, floor, ceil
hypot(3, 4), atan2(1, 0)          # 5.0, pi/2 radians
```

`comb`/`factorial` give full integers, not values modulo `MOD`. Use `isqrt`, not float `sqrt`, for exact roots.

### 2.3 Exact fractions

```
from fractions import Fraction as F
x = F(1, 3) + F(1, 6)            # Fraction(1, 2)
x.numerator, x.denominator       # (1, 2)
F("0.125")                       # 1/8, exact decimal
F(2, 3) < F(3, 4)                # exact comparison
F("3.14159").limit_denominator(100)  # 311/99
```

Fractions reduce automatically. Use integers or strings: `F(0.1)` preserves a binary float, not exactly `1/10`.

### 2.4 High-precision decimal arithmetic

```
from decimal import Decimal as D, localcontext
with localcontext() as ctx:
    ctx.prec = 60                # significant digits
    x = D(1) / D(7)              # rounded to precision
    root = D(2).sqrt()
    y = D("0.1") + D("0.2")      # exactly 0.3
    z = D("1.235").quantize(D("0.01"))  # 1.24
```

Use strings for decimal input. Context controls precision and rounding. For exact `1/3`, use `Fraction`.

### 2.5 Integer bitmasks

```
mask, i, n = 0b10110, 1, 6
mask.bit_count(), mask.bit_length()  # 3, 5
bool(mask & (1 << i))            # test bit i
mask |= 1 << 5                   # set bit 5
mask &= ~(1 << 1)                # clear bit 1
mask ^= 1 << 2                   # toggle bit 2
mask << 3, mask >> 1             # unlimited-width shifts
(~mask) & ((1 << n) - 1)         # complement in n bits
```

## 3 Containers

### 3.1 Dictionaries and composite keys

```
d = {"a": 2}
d["b"] = 3
d.get("missing", 0)              # 0; does not insert
"a" in d                         # membership tests keys
list(d.items())                  # [('a',2),('b',3)]
list(d.keys()), list(d.values())
d.pop("missing", None)           # default avoids KeyError
positions = {(2, 3): 7}          # tuple key, built-in hash
frozen = frozenset([1, 2, 2])    # hashable set: {1,2}
```

Dict iteration preserves insertion order. Keys must be hashable: use tuples/frozensets, not lists/sets. Missing `d[k]` raises `KeyError`.

### 3.2 Sets

```
s, t = {1, 2, 3}, {3, 4}
s | t, s & t                     # union, intersection
s - t, s ^ t                     # difference, symmetric diff
s <= t, s.isdisjoint(t)          # subset, no overlap
s.add(5); s.discard(99)          # discard tolerates missing
empty = set()                    # {} is an empty dict
unique = set([3, 1, 3])
```

Sets have no sorted-order guarantee. Set/dict membership is average $O(1)$; list membership is $O(n)$.

### 3.3 Frequency tables and multisets

```
from collections import Counter
c = Counter("banana")            # a:3, n:2, b:1
c["x"]                           # 0; does not insert
c.most_common(2)                 # [('a',3), ('n',2)]
list(c.elements())               # repeat by multiplicity
c + Counter("an"), c - Counter("an")
c & Counter("band"), c | Counter("band")
# &: minimum counts; |: maximum; +/-: add/subtract
```

Counter arithmetic drops nonpositive counts. `c.subtract(other)` mutates and keeps zero/negative counts.

### 3.4 Grouping with missing defaults

```
from collections import defaultdict
groups = defaultdict(list)
groups["a"].append(4)            # creates [] if missing
counts = defaultdict(int)
counts["a"] += 1                 # creates 0 if missing
sets = defaultdict(set)
sets["a"].add(4)
```

### 3.5 Deque: queue and both ends

```
from collections import deque
q = deque([1, 2])                # O(1) ends; list.pop(0): O(n)
q.append(3); q.appendleft(0)
q.popleft(), q.pop()             # 0, 3
q.rotate(1)                      # [2,1]: right by 1
q[0], q[-1]                      # 2, 1
```

## 4 Strings and order

### 4.1 Strings

```
s = "  one  two\n"
s.strip(), s.split()             # 'one  two', ['one','two']
"a,,b,".split(",")               # ['a','','b','']
"-".join(["a", "b", "c"])        # 'a-b-c'
s.replace("one", "1")            # replace all occurrences
"banana".find("na")              # 2; -1 if absent
"banana".count("an")             # 2, non-overlapping
"abc".startswith("ab"), "abc".endswith("c")
"abc"[::-1], "AbC".lower()       # 'cba', 'abc'
ord("a"), chr(97)                # 97, 'a'
```

Strings are immutable. Build many output fragments in a list, then join; repeated concatenation can be quadratic. `split()` collapses whitespace; `split(",")` keeps empty fields.

### 4.2 Formatting and conversions

```
x = 1 / 3
f"{x:.10f}"                      # '0.3333333333'
f"{7:03d}", f"{255:x}"           # '007', 'ff'
f"{13:08b}"                      # '00001101'
"  abc  ".strip()                # 'abc'
int("-42"), float("1.25")        # -42, 1.25
str(123), list("abc")            # '123', ['a','b','c']
```

### 4.3 Sorting and key functions

```
a = [(2, 5), (1, 7), (2, 3)]
b = sorted(a)                    # lexicographic tuples
a.sort(key=lambda p: (p[0], -p[1]))
sorted(["bbb", "a", "cc"], key=len)
sorted([3, 1, 2], reverse=True)  # [3,2,1]
max(a, key=lambda p: p[1])       # item with largest second
min([], default=0)               # 0 instead of ValueError
```

Sorts are stable; equal keys keep their original order. `a.sort()` mutates and returns `None`; `sorted(a)` makes a new list.

### 4.4 Binary search in a sorted list

```
from bisect import bisect_left as bl, bisect_right as br
from bisect import insort
a, x = [1, 2, 2, 5], 2
bl(a, x), br(a, x)               # 1, 3
count = br(a, x) - bl(a, x)      # 2 occurrences
insort(a, 3)                     # [1,2,2,3,5]
```

Search: $O(\log n)$. Insertion: $O(n)$. This is not a balanced ordered set.

### 4.5 Extracting structured text

```
import re
re.findall(r"-?\d+", "x=-12, y=34")  # ['-12','34']
re.fullmatch(r"[A-Z]+\d+", "AB12") is not None
re.split(r"[,;]+", "a,b;;c")     # ['a','b','c']
```

## 5 Iteration and helpers

### 5.1 Combinatorial iterators

```
from itertools import (combinations, permutations, product,
    combinations_with_replacement, accumulate, chain, pairwise)
list(combinations("ABC", 2))     # AB, AC, BC (as tuples)
list(permutations("ABC", 2))     # AB, AC, BA, BC, CA, CB
list(product([0, 1], repeat=3))  # 000, 001, ..., 111
list(product("AB", [1, 2]))      # A1, A2, B1, B2
list(combinations_with_replacement("AB", 2))  # AA, AB, BB
list(accumulate([2, 3, 4], initial=0))  # [0,2,5,9]
list(chain.from_iterable([[1, 2], [3]]))  # [1,2,3]
list(pairwise([4, 7, 9]))        # [(4,7),(7,9)]
```

These return lazy iterators; `list(...)` above shows the results. Iterate directly to avoid storing all outputs. Repeated input values can give duplicate permutations.

### 5.2 Automatic memoization

```
from functools import cache

@cache
def f(i, mask):
    ...                          # translate the recurrence

f.cache_clear()                  # between different test cases
# Older Python: @lru_cache(None), from functools import lru_cache
```

Arguments must be hashable (e.g. ints, tuples, frozensets). Cache keys include arguments, not changing global variables. Clear between cases or make a new function. Recursion depth still applies; iterative traversal is safer for deep graphs.

### 5.3 Min-heaps and sorted merging

```
from heapq import (heapify, heappush, heappop, heappushpop,
                   heapreplace, nsmallest, nlargest, merge)
h = [5, 2, 8]
heapify(h)                       # O(n); minimum at h[0]
heappush(h, 3)
heappop(h)                       # 2; O(log n)
heappushpop(h, 4)                # push, then pop minimum
heapreplace(h, 6)                # pop minimum, then push
nlargest(2, [4, 1, 7]), nsmallest(2, [4, 1, 7])
list(merge([1, 4], [2, 3]))      # [1,2,3,4]; inputs sorted
```

Min-heap: use `(-priority, item)` for a max-heap. Tuples compare lexicographically; add a unique counter before non-comparable payloads. `heapreplace` needs a nonempty heap. Merging is lazy.

### 5.4 Comprehensions and generators

```
a = [3, 1, 3]
index = {x: i for i, x in enumerate(a)}  # last index of x
pairs = {(x, y) for x in range(2) for y in range(3)}
total = sum(x*x for x in a)      # generator: no extra list
first, *middle, last = [1, 2, 3, 4]
```

Use brackets for a list, braces for a set/dict, `(expr for ...)` for a generator.

## 6 Dates and reminders

### 6.1 Calendar arithmetic

```
from datetime import date, timedelta
a, b = date(2024, 2, 28), date(2024, 3, 1)
(b - a).days                     # 2 (leap year)
a + timedelta(days=2)            # 2024-03-01
a.weekday()                      # 2: Monday=0, Sunday=6
a.isoformat()                    # '2024-02-28'
date.fromisoformat("2024-03-01")  # date(2024,3,1)
from calendar import isleap, monthrange
isleap(2024), monthrange(2024, 2)[1]  # True, 29 days
```

`date` supports years 1–9999. `monthrange(y,m)` returns (weekday of day 1, days in month).

### 6.2 Topological order

```
from graphlib import TopologicalSorter, CycleError
pred = {"A": set(), "B": {"A"}, "C": {"A", "B"}}
order = list(TopologicalSorter(pred).static_order())
# ['A','B','C']; CycleError if the graph has a cycle
```

Input lists **predecessors**, not outgoing neighbours. With several valid orders, the returned order need not be lexicographically smallest.

### 6.3 JSON input

```
import json
obj = json.loads('{"a": [1, 2], "ok": true}')
obj["a"][0], obj["ok"]           # 1, True
json.dumps(obj)                  # encode back to JSON
```

### 6.4 C++ to Python: avoid surprises

- Indentation defines blocks; use 4 spaces.
- and/or/not, True/False/None; no ++.
- append/sort/reverse mutate and return None.
- == compares values; is tests identity. Use `x is None` for a missing value.
- `(x,)` is a one-element tuple; `(x)` is just `x`.
- Iterating a dict gives keys. Do not change a dict/set's size while iterating it.
- Integers do not overflow; floats still round.

### 6.5 Complexities to remember

| Operation | Cost |
|---|---|
| List index / last append or pop | $O(1)$ amort. |
| List search / front or middle insert | $O(n)$ |
| Dict/set lookup, add, remove | $O(1)$ avg. |
| Deque append/pop at either end | $O(1)$ |
| Heap push/pop | $O(\log n)$ |
| Sort | $O(n \log n)$ |
| List/string slice of length $k$ | $O(k)$ |

Bounds above treat numbers/keys as fixed-size. Big integer arithmetic and hashing long keys depend on their size.

### Integer equations

**N1 Solve $ax + by = c$ in integers**

*Symbols:* $a, b, c$ given integers, $(a, b) \neq (0, 0)$; $g = \gcd(a, b) > 0$ must divide $c$; $(x_0, y_0)$ one solution; $t$ any integer. All solutions:

$$x = x_0 + (b/g)t, \qquad y = y_0 - (a/g)t.$$

For bounds $L \le x_0 + ht \le R$, $h > 0$: $\lceil (L - x_0)/h \rceil \le t \le \lfloor (R - x_0)/h \rfloor$. Intersect $t$ ranges for $x, y$.

### Modulo and totients

Moduli are positive integers; exponents are integers $e \ge 0$.

**N2 Count integers coprime to $n$: totient**

*Symbols:* $n = \prod_i p_i^{e_i} \ge 1$; distinct primes $p_i$, exponents $e_i \ge 1$; $d$ runs over positive divisors.

$\varphi(n) = \#\{1 \le x \le n : \gcd(x, n) = 1\}$; $\varphi(1) = 1$.

$$\varphi(n) = n \prod_{p \mid n} (1 - 1/p) = \prod_i p_i^{e_i - 1}(p_i - 1).$$

$\varphi(p^k) = p^{k-1}(p - 1)$ for prime $p$, $k \ge 1$; $\sum_{d \mid n} \varphi(d) = n$.
$\gcd(a, b) = 1 \Rightarrow \varphi(ab) = \varphi(a)\varphi(b)$.

**N3 Invert / reduce a power: Euler, Fermat**

*Symbols:* $a$ integer base, $m > 1$ modulus, $p$ prime; $a^{-1}$ is a value whose product with $a$ is 1 modulo $m$.

**Require $\gcd(a, m) = 1$:**

$$a^{\varphi(m)} \equiv 1, \qquad a^e \equiv a^{e \bmod \varphi(m)} \pmod m.$$

Inverse: $a^{-1} \equiv a^{\varphi(m)-1} \pmod m$. Prime $p$: $a^p \equiv a \pmod p$ always; $a^{p-1} \equiv 1 \pmod p$ if $p \nmid a$.

**N4 Solve / cancel congruences; merge by CRT**

*Symbols:* $a, b$ known integers; $m, n$ moduli; $x, y$ unknown integers; $g = \gcd(a, m)$; $x_0$ one solution.

$ax \equiv b \pmod m$: no solution if $g \nmid b$. Otherwise solve $(a/g)x \equiv b/g \pmod{m/g}$: $x = x_0 + t(m/g)$, $t = 0, \ldots, g - 1$: $g$ residues modulo $m$.

$$ax \equiv ay \pmod m \iff x \equiv y \pmod{m/g}.$$

CRT: $x \equiv a \pmod m$, $x \equiv b \pmod n$ are compatible iff $\gcd(m, n) \mid (b - a)$; period $\operatorname{lcm}(m, n)$.

**N5 Power period / noncoprime bases**

*Symbols:* $d = \operatorname{ord}_m(a)$: smallest positive exponent giving 1; $q$ prime divisor of $d$; $p^k$ prime-power modulus.

For $m > 1$, $\gcd(a, m) = 1$: $d \mid \varphi(m)$. Start $d = \varphi(m)$; repeatedly divide by $q$ if $a^{d/q} \equiv 1 \pmod m$.

For $a \ge 1$, write $a = p^s u$, $p \nmid u$:

$$a^e \equiv \begin{cases} 0, & se \ge k, \\ p^{se} u^e, & se < k \end{cases} \pmod{p^k}.$$

$s$ counts factors of $p$ in $a$; reduce only $u$'s exponent; merge prime powers by CRT.

### Divisors and gcd sums

**D1 Undo divisor sums: Möbius inversion**

*Symbols:* $n \ge 1$ integer; $d$ positive divisor; $r$ distinct prime-factor count; $f, F$ arithmetic functions.

$\mu(1) = 1$; $\mu(n) = 0$ if any prime square divides $n$; otherwise $\mu(n) = (-1)^r$. Also $\sum_{d \mid n} \mu(d) = [n = 1]$.

$$F(n) = \textstyle\sum_{d \mid n} f(d) \iff f(n) = \sum_{d \mid n} \mu(d) F(n/d).$$

Multiplicative $f$: $f(1) = 1$ and $f(ab) = f(a)f(b)$ for coprime $a, b$; compute from the prime powers in $n$.

### D2 Count coprime values / pairs; sum gcds

*Symbols:* $N, M, m \geq 1$ integers; $(x, y)$ ordered pair with $1 \leq x \leq N$, $1 \leq y \leq M$; $h_d = \lfloor N/d \rfloor \lfloor M/d \rfloor$.

$$\#\{1 \leq x \leq N : \gcd(x, m) = 1\} = \textstyle\sum_{d|m} \mu(d) \lfloor N/d \rfloor.$$
$$\#\{\gcd(x, y) = 1\} = \textstyle\sum_{d \leq \min(N,M)} \mu(d) h_d,$$
$$\textstyle\sum_{x=1}^{N} \sum_{y=1}^{M} \gcd(x, y) = \sum_{d \leq \min(N,M)} \varphi(d) h_d.$$

For gcd exactly $g \geq 1$, use the coprime-pair formula with $N \leftarrow \lfloor N/g \rfloor$, $M \leftarrow \lfloor M/g \rfloor$.

### D3 Group floors / count $xy \leq N$

*Symbols:* $N \geq 1$ bound; $l, r$ first/last index of a block; $q$ common quotient; $s = \lfloor \sqrt{N} \rfloor$; $\tau(k)$ divisor count; $x, y$ positive integers, ordered.

$$q = \lfloor N/l \rfloor, \qquad r = \lfloor N/q \rfloor.$$

Then $\lfloor N/i \rfloor = q$ for $l \leq i \leq r$. Start $l = 1$; advance $l = r + 1$: $O(\sqrt{N})$ blocks.

$$\textstyle\sum_{i=1}^{N} \lfloor N/i \rfloor = \sum_{k=1}^{N} \tau(k) = 2 \sum_{i=1}^{s} \lfloor N/i \rfloor - s^2.$$

## Graph counts

*Symbols:* Simple undirected graph: no loops/parallel edges. $V, E$: vertex/edge sets; $n = |V|$; $m = |E|$; $c$: components; $d_v$: degree. $\#C_k$: simple $k$-cycles, rotations/reversals identified.

### G1 Independent cycles / degrees / cuts

$$r = m - n + c, \qquad \sum_v d_v = 2m.$$

*Symbols:* $r$: independent cycles, or edges to remove for a spanning forest. Even-degree edge subsets: $2^r$, including empty. $r$ is not the total simple-cycle count. Odd-degree vertex count is even. A cut separates a vertex subset from its complement; cycles cross it evenly. Cut-space dimension: $n - c$. Bridges belong to no cycle.

### G2 Triangles

*Symbols:* $N(v)$: neighbours; $a_{uv} = |N(u) \cap N(v)|$: common-neighbour count. $A$: $0/1$ adjacency matrix, diagonal 0. $\mathrm{tr}$: diagonal sum.

$$\#C_3 = \frac{1}{3} \sum_{\{u,v\} \in E} a_{uv} = \frac{\mathrm{tr}(A^3)}{6}.$$

### G3 Four-cycles / walks

$$\#C_4 = \tfrac{1}{2} \textstyle\sum_{u<v} \binom{a_{uv}}{2},$$
$$= \frac{\mathrm{tr}(A^4) + 2m - 2\sum_v d_v^2}{8}.$$

$u < v$: unordered vertex pairs. Bipartite: sum pairs in one side only, omit $1/2$. Chords allowed. $(A^k)_{uv}$: length-$k$ walks, allowing repetitions. $\mathrm{tr}(A^k)/(2k)$ is not a general simple-cycle formula.

### G4 Complete-graph cycles

*Symbols:* $K_n$: every pair joined; $K_{a,b}$: complete bipartite, side sizes $a, b$. Ranges: $3 \leq k \leq n$ and $2 \leq k \leq \min(a, b)$, respectively.

$$\#C_k(K_n) = \tfrac{n!}{2k(n-k)!},$$
$$\#C_{2k}(K_{a,b}) = \tbinom{a}{k} \tbinom{b}{k} \tfrac{k!(k-1)!}{2}.$$

Hamilton cycles (all vertices): $(n-1)!/2$, $n \geq 3$. Complete directed graph with both arcs per pair: replace $2k$ by $k$ for $k \geq 3$; reversal is distinct. Labeled simple graphs on $n$ vertices: $2^{\binom{n}{2}}$.

## Trees and planar graphs

### G5 Labeled trees / prescribed degrees

*Symbols:* Labels distinguish vertices; $n \geq 2$. $d_i \geq 1$: required degree of label $i$; $\sum_i d_i = 2n - 2$.

Cayley: $n^{n-2}$ labeled trees; $n^{n-1}$ with a chosen root. Both counts are 1 for $n = 1$.

$$\#\text{trees with degrees } d_i = \frac{(n-2)!}{\prod_i (d_i - 1)!}.$$

Prüfer sequence: $n - 2$ labels; label $i$ occurs $d_i - 1$ times. Leaves are exactly absent labels. For $n \geq 2$:

$$\#\text{leaves} = 2 + \sum_{v : d_v \geq 3} (d_v - 2).$$

### G6 Spanning trees (Matrix–Tree)

*Symbols:* $\tau(G)$: spanning-tree count; $C_n$: one $n$-cycle. Laplacian $L$: $L_{ii} = d_i$; $L_{ij} = -1$ if adjacent, 0 otherwise. $L^{(j)}$: remove row and column $j$; $\det$: determinant.

$$\begin{aligned} \tau(G) &= \det L^{(j)}, \\ \tau(K_n) &= n^{n-2} && (n \geq 2), \\ \tau(K_{a,b}) &= a^{b-1} b^{a-1} && (a, b \geq 1), \\ \tau(C_n) &= n && (n \geq 3). \end{aligned}$$

Disconnected $G$: 0. Full $\det L = 0$. Elimination modulo a prime: $O(n^3)$; row swaps flip the sign. Counts all spanning trees, not only minimum-weight trees.

### G7 Planar faces / edge bounds

*Symbols:* $f$: faces including outside; planar means drawable without crossings.

$$n - m + f = 1 + c.$$

For simple planar graphs, $n \geq 3$:

$$m \leq 3n - 6; \qquad m \leq 2n - 4 \text{ if bipartite.}$$

Bounds do not prove planarity. Triangulation (all faces triangles): $m = 3n - 6$, $f = 2n - 4$. Every nonempty simple planar graph has a vertex of degree at most 5.

## Partitions and symmetry

### C1 Partitions / onto functions

*Symbols:* $n$: distinct objects; $k$: groups/cycles. $c(n, k)$: permutations with $k$ cycles (unsigned Stirling I). $S(n, k)$: $k$ nonempty unlabeled groups (Stirling II). Bell $B_n = \sum_k S(n, k)$: all partitions.

$$c(n, k) = c(n-1, k-1) + (n-1)c(n-1, k),$$
$$S(n, k) = S(n-1, k-1) + kS(n-1, k).$$

Both $(0, 0) = 1$; other $k = 0$ or $k > n$ entries: 0. Use $0^0 = 1$.

$$k!S(n, k) = \textstyle\sum_{j=0}^{k} (-1)^j \binom{k}{j} (k - j)^n.$$

Onto: $n$ objects map to $k$ labeled targets, all used; $j$: excluded targets in inclusion–exclusion.

### C2 Prescribed permutation cycles

*Symbols:* $a_j \geq 0$: cycles of length $j$; $\sum_j j a_j = n$. $H_n = \sum_{j=1}^{n} 1/j$.

$$\#\text{permutations} = \frac{n!}{\prod_j j^{a_j} a_j!}.$$

Uniform random permutation: expected $j$-cycles $= 1/j$ for $1 \leq j \leq n$; expected total cycles $= H_n$.

### C3 Symmetric objects (Burnside)

*Symbols:* $X$: objects; $G$: finite group of allowed symmetries. $\mathrm{Fix}(g)$: objects unchanged by symmetry $g$. Orbit: objects equivalent under the symmetries.

$$\#\text{orbits} = \frac{1}{|G|} \sum_{g \in G} |\operatorname{Fix}(g)|.$$

Do not just divide $|X|$ by $|G|$. A position permutation with $z$ cycles fixes $q^z$ colorings with $q$ colors.

### C4 Necklaces / bracelets

*Symbols:* $n \geq 1$: positions; $q$: colors; $r$: rotation offset; $d$: divisor of $n$. $R, F$: total fixed colorings over rotations, reflections; $\varphi$: totient (N2).

$$R = \textstyle\sum_{r=0}^{n-1} q^{\gcd(n,r)} = \sum_{d|n} \varphi(d) q^{n/d}.$$

Necklaces: $R/n$ (rotations equivalent). Bracelets: $(R + F)/(2n)$ (also reflections).

$$F = \begin{cases} nq^{(n+1)/2}, & n \text{ odd}, \\ \frac{n}{2}(q^{n/2} + q^{n/2+1}), & n \text{ even}. \end{cases}$$

Modulo prime $p$: invert denominator $d = n$ or $2n$ only if $p \nmid d$; otherwise compute numerator modulo $pd$ and integer-divide the residue by $d$ (B5).

## Sums and sequences

### S1 Weighted geometric sums

*Symbols:* $n \geq 0$: integer upper limit; $i$: index; $r$: ratio, $r \neq 1$.

$$\sum_{i=1}^{n} ir^i = \frac{r(1-(n+1)r^n+nr^{n+1})}{(1-r)^2}.$$

At $r = 1$: $n(n+1)/2$. For $|r| < 1$: $\sum_{i\geq 1} ir^i = r/(1-r)^2$. Modulo $m > 0$: denominator must be invertible.

### S2 Pair distances and products

*Symbols:* $a_1, \ldots, a_n$: real values; $i < j$: each pair once. First formula: sort $a_1 \leq \cdots \leq a_n$.

$$\sum_{i<j} |a_i - a_j| = \sum_{i=1}^{n} (2i - n - 1)a_i.$$

Without sorting:

$$\sum_{i<j} (a_i - a_j)^2 = n \sum_i a_i^2 - \left(\sum_i a_i\right)^2,$$

$$\sum_{i<j} a_i a_j = \frac{(\sum_i a_i)^2 - \sum_i a_i^2}{2}.$$

Manhattan distances: apply the first formula per coordinate.

### S3 Fibonacci doubling and sums

*Symbols:* $F_0 = 0$, $F_1 = 1$, $F_{n+2} = F_{n+1} + F_n$; $n, k \geq 0$: integer indices.

$$F_{2k} = F_k(2F_{k+1} - F_k),$$
$$F_{2k+1} = F_k^2 + F_{k+1}^2.$$

$\sum_{i=0}^{n} F_i = F_{n+2} - 1$; $\sum_{i=0}^{n} F_i^2 = F_n F_{n+1}$. $\gcd(F_a, F_b) = F_{\gcd(a,b)}$ for integer $a, b \geq 1$. Doubling takes $O(\log n)$ arithmetic steps. Modulo $m$: all identities work except the gcd identity.

### S4 Convolution and recurrences

*Symbols:* $A(x) = \sum_{n\geq 0} a_n x^n$, $B(x) = \sum_{n\geq 0} b_n x^n$; $[x^n]$: coefficient of $x^n$; $x$: formal variable.

$$[x^n](AB) = \sum_{i=0}^{n} a_i b_{n-i}.$$

For constants $c, d$ and $a_n = ca_{n-1} + da_{n-2}$, $n \geq 2$:

$$A(x) = \frac{a_0 + (a_1 - ca_0)x}{1 - cx - dx^2}.$$

Formal series: no convergence requirement.

## Binomials and modular counting

*Symbols:* $\binom{n}{r}$: choose $r$ of $n$ objects; parameters and indices are integers. Upper arguments are nonnegative; $\binom{n}{r} = 0$ outside $0 \leq r \leq n$.

### B1 Binomial sums

For nonnegative integer parameters:

$$\sum_{i=0}^{n} \binom{i}{k} = \binom{n+1}{k+1},$$
$$\sum_k \binom{a}{k}\binom{b}{r-k} = \binom{a+b}{r},$$
$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n},$$
$$\sum_{k=0}^{n} k\binom{n}{k} = n\,2^{n-1} \quad (n \geq 1),$$
$$\sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0 \quad (n \geq 1).$$

### B2 Prime factors and trailing zeros

*Symbols:* $p$: prime; $v_p(N)$: exponent of $p$ in integer $N > 0$; $0 \leq r \leq n$ (Legendre).

$$v_p(n!) = \sum_{j\geq 1} \lfloor n/p^j \rfloor,$$
$$v_p\tbinom{n}{r} = v_p(n!) - v_p(r!) - v_p((n-r)!).$$

Base $b = \prod_i p_i^{e_i} > 1$, with distinct primes $p_i$:

$$\text{Trailing zeros of } n! = \min_i \lfloor v_{p_i}(n!)/e_i \rfloor.$$

### B3 Binomial mod a prime (Lucas)

*Symbols:* $p$: prime modulus; $n_i, r_i$: base-$p$ digits, $n = \sum_i n_i p^i$, $r = \sum_i r_i p^i$, $0 \leq n_i, r_i < p$.

$$\binom{n}{r} \equiv \prod_i \binom{n_i}{r_i} \pmod{p}.$$

Any $r_i > n_i$ gives 0. Factorial inverses require indices $< p$; $p!$ has no inverse modulo $p$.

### B4 Binomial mod a prime power

*Symbols:* $p$: prime; $k \geq 1$; $0 \leq r \leq n$; $e = v_p\binom{n}{r}$; $U(t) = t!/p^{v_p(t!)}$.

If $e \geq k$, answer 0. Otherwise:

$$\binom{n}{r} \equiv p^e U(n)U(r)^{-1}U(n-r)^{-1} \pmod{p^k}.$$

Strip $p$ factors before reducing/inverting $U$ modulo $p^k$. Combine prime powers by CRT.

### B5 Exact modular division

*Symbols:* $A$: integer numerator; $d > 0$: divisor with $d \mid A$; $m > 0$: modulus; $A \bmod (dm) \in [0, dm)$.

$$(A/d) \bmod m = \frac{A \bmod (dm)}{d}.$$

Integer-divide the residue by $d$; no inverse needed.

## Probability and expectation

*Symbols:* $X, Y, X_i$: random variables; $P$: probability; $E$: expectation; $\mathrm{Var}$: variance; $\mathrm{Cov}$: covariance. All required moments finite.

### P1 Expectation and variance

*Symbols:* $a, b$: constants; $I \in \{0, 1\}$: event indicator, $P(I = 1) = p$.

$E[\sum_i X_i] = \sum_i E[X_i]$, without independence.

$$\mathrm{Var}(X) = E[X^2] - E[X]^2,$$
$$\mathrm{Cov}(X, Y) = E[XY] - E[X]E[Y].$$

$$\mathrm{Var}\left(\sum_i X_i\right) = \sum_i \mathrm{Var}(X_i) + 2\sum_{i<j} \mathrm{Cov}(X_i, X_j).$$

$\mathrm{Var}(aX + b) = a^2 \mathrm{Var}(X)$. Independence gives covariance 0.

$E[I] = p$, $\mathrm{Var}(I) = p(1-p)$.

### P2 Tail sums and extrema

For nonnegative integer-valued $X$:

$$E[X] = \sum_{k\geq 1} P(X \geq k),$$
$$E[X^2] = \sum_{k\geq 1} (2k-1)P(X \geq k).$$

For independent $X_i$, real threshold $t$:

$$P(\max_i X_i \leq t) = \prod_i P(X_i \leq t),$$
$$P(\min_i X_i > t) = \prod_i P(X_i > t).$$

### P3 Independent trials: binomial

*Symbols:* $n$ independent trials, success probability $p \in [0, 1]$; $X$: success count; $k$: outcome, $0 \leq k \leq n$.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

$E[X] = np$; $\mathrm{Var}(X) = np(1-p)$.

### P4 First success: geometric

*Symbols:* Independent trials, success probability $p \in (0, 1]$; $X$: trials through first success; $k$: outcome, $k \geq 1$.

$$P(X = k) = (1-p)^{k-1} p.$$

$E[X] = 1/p$; $\mathrm{Var}(X) = (1-p)/p^2$.

### P5 No replacement: hypergeometric

*Symbols:* $N > 1$ objects, $0 \leq K \leq N$ marked; $0 \leq n \leq N$ uniform draws without replacement; $X$: marked count; $k$: outcome; $p = K/N$.

$$P(X = k) = \frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}.$$

$E[X] = np$; $\mathrm{Var}(X) = np(1-p)(N-n)/(N-1)$.

### P6 Probability bounds

*Symbols:* $t > 0$: threshold; $A_i$: events.

Markov ($X \geq 0$): $P(X \geq t) \leq E[X]/t$. Chebyshev: $P(|X - E[X]| \geq t) \leq \mathrm{Var}(X)/t^2$. Union: $P(\bigcup_i A_i) \leq \sum_i P(A_i)$; no independence needed.